# Reviewing and retrying skipped records

Regional ingestion is source intake only — never automatic verification or publication. As a page of records is read, individual records can be **rejected** by validation. This note explains how a rejected record is recorded, how to review one, how a retry happens, and how long the review material is kept.

## Two different “skips”, handled differently

| | **Rejected record** | **Failed write** |
|---|---|---|
| Cause | Permanent validation failure (wrong county, missing taxon, withheld source, malformed feature) | Transient database write error while saving an otherwise valid record |
| Page status | `partial` | `partial` |
| Effect on cursor | Cursor **advances** past it — the record will never validate as-is, so holding would stall the whole county | Cursor is **held** — the write is idempotent and is retried on the next run |
| Surfaced as | `rejected` count + `rejections[]` entries | `failedWrites` count |

The cursor only advances when a run reports an **acknowledged write pass** ( `writesAcknowledged: true` ,
i.e. a confirmed non-dry-run pass) **and** `failedWrites === 0` . A dry run, a missing acknowledgement, or any non-zero (or absent) `failedWrites` holds every cursor.

## Fixed rejection reason codes

A rejected record is only ever tagged with one of a small, fixed allowlist of developer-authored reason codes. Any other error — an upstream response body, free text, coordinates, or personal data —
collapses to the generic code `Rejected by validation` , so nothing sensitive can reach the report.

| Reason code | Meaning |
|---|---|
| `Wrong publisher or county` | Record’s publisher or GADM county does not match the requested county |
| `Wrong authority` | Planning record’s authority does not match the requested authority |
| `Withheld/generalised source needs separate review` | Upstream marked the source as withheld or generalised |
| `Missing taxon` | Record has no usable taxon/scientific name |
| `Missing source identity` | Record has no usable stable identifier |
| `Invalid source object` | Record is not a well-formed source object |
| `Source feature too large` | Serialised feature exceeds the per-record size limit |
| `Rejected by validation` | Generic fallback for any reason outside the allowlist |

The allowlist lives in `lib/ingest/connectors-scotland.ts` (`REJECTION_REASONS`). Add a new code there deliberately when a connector introduces a new validation throw; an un-listed message is reported as the generic code rather than echoed.

## Where rejections surface

**API response** (`/api/ingest/external`, HTTP `207` when a page is partial): a top-level `rejected` count, and for each source a `rejections[]` array of `{ locator, reason }`, where `locator` is the record identifier (best-effort) and `reason` is one of the codes above. No record bodies are included.

**Refresh report** (`report.json`, written by `scripts/regional-refresh.mjs` into the state directory): one entry per source with `status`, `rejected`, `rejections`, `acknowledged` (the run’s `writesAcknowledged`), `failedWrites`, and the resulting `nextOffset`. The scheduled run also exits with code `2` whenever any source is non-`ok` or has failed writes, so a partial page is a visible review flag rather than a silent pass.

## How to review

1. Read `report.json` from the state directory (or inspect the `207` API response directly).
2. For each source with `rejected > 0`, read its `rejections[]`: the `reason` tells you the class of failure, the `locator` identifies which upstream record.
3. Cross-check the reason against the table above to decide whether the rejection is expected (e.g. a genuinely out-of-county record) or a connector/data issue to fix.

## How to retry

- **Rejected records** are skipped on purpose so the cursor can advance. They are re-read on the next
  full sweep: when a source reaches the end of its records the cursor resets to `0`, so the next sweep re-reads offset `0` and re-evaluates every record. To make a previously-rejected record pass, fix the upstream data or adjust the connector’s validation rule, then let the next sweep pick it up. No manual cursor edit is needed.
- **Failed writes** need no manual action: the cursor is held, `receive` is idempotent, and the same page is retried automatically on the next run until `failedWrites === 0`.
- To force an immediate re-read of a specific source, delete its entry from `cursors.json` in the state directory; the next run starts that source from offset `0`.

## Report retention

- `report.json` is **overwritten on every run** — it is always the latest snapshot, not a history. If you need a durable audit trail, copy or ship `report.json` to external storage after each run (e.g. archive it by timestamp); the `checkedAt` field records when the snapshot was taken.
- `cursors.json` persists across runs and is the durable per-source position; it is written atomically after every source.
- Neither file stores record bodies, upstream response bodies, credentials, or personal data — only counts, fixed reason codes, record locators, and cursor offsets. Retaining them carries no sensitive data.